

METİN GÖMME (TEXT EMBEDDINGS) UYGULAMALARI

Google Cloud Vertex AI ve Makine Öğrenmesi ile Geliştirici Çalışma Raporu

1. Metin Gömme (Text Embeddings) Nedir?

Metin gömme (text embeddings), yapay zekanın karmaşık dilleri bilgisayarların kolayca işleyebileceği sayısal vektörlere dönüştürme işlemidir. Bu teknikle, kelimeler veya paragraflar birer sayı dizisi olarak temsil edilir ve benzer anlamlara sahip ifadeler vektör uzayında birbirine yakın konumlanır. Bu çalışmada kullanılan **textembedding-gecko** modeli, her metin girdisi için **768 boyutta** bir vektör üretir. Bu 768 boyutlu vektörler, metnin anlamsal derinliğini, bağlamını ve dilbilgisel yapısını eksiksiz bir şekilde temsil eder.

Günlük Hayat Analogisi: Anlamsal Formlar

Düşünün ki her kitabın anlamsal içeriğini tanımlayan 768 maddelik bir formumuz var. Kitapları doğrudan okuyup karşılaştırmak yerine, form üzerindeki puanların birbirine ne kadar benzediğine bakarak kitapların benzerliğini milisaniyeler içinde ölçebiliriz.

2. Veri Hazırlığı ve API İsteklerinin Yönetimi

Çalışmada kullanılan Stack Overflow veri seti, 4 farklı yazılım dili (Python, HTML, R vb.) ile ilgili toplam **2.000 adet soru ve kabul edilmiş yanıt**tan oluşmaktadır. Model API'si tek bir istekte maksimum **5 adet metin girdisi** işleyebildiği için veri setinin demetler (batches) halinde API'ye gönderilmesi gerekir. Ayrıca Google Cloud Vertex AI rate limit (istek hızı sınırı) engeline takılmamak için demetler arası bekleme sürelerini yöneten özel yardımcı işlevler kullanılır.

```
# 1. Vertex AI ve Gömme Modelinin İklendirilmesi import vertexai from vertexai.language_models import
TextEmbeddingModel vertexai.init(project="gcp-projeniz", location="us-central1") model =
TextEmbeddingModel.from_pretrained("textembedding-gecko") # 2. İstek Limiti İçin Demetleme Yardımcı
Fonksiyonu def generate_batches(data, batch_size=5): for i in range(0, len(data), batch_size): yield
data[i:i + batch_size] # 200 satırlık veriyi 5'li demetlere bölme batches =
list(generate_batches(df['input_text'][:200], batch_size=5))
```

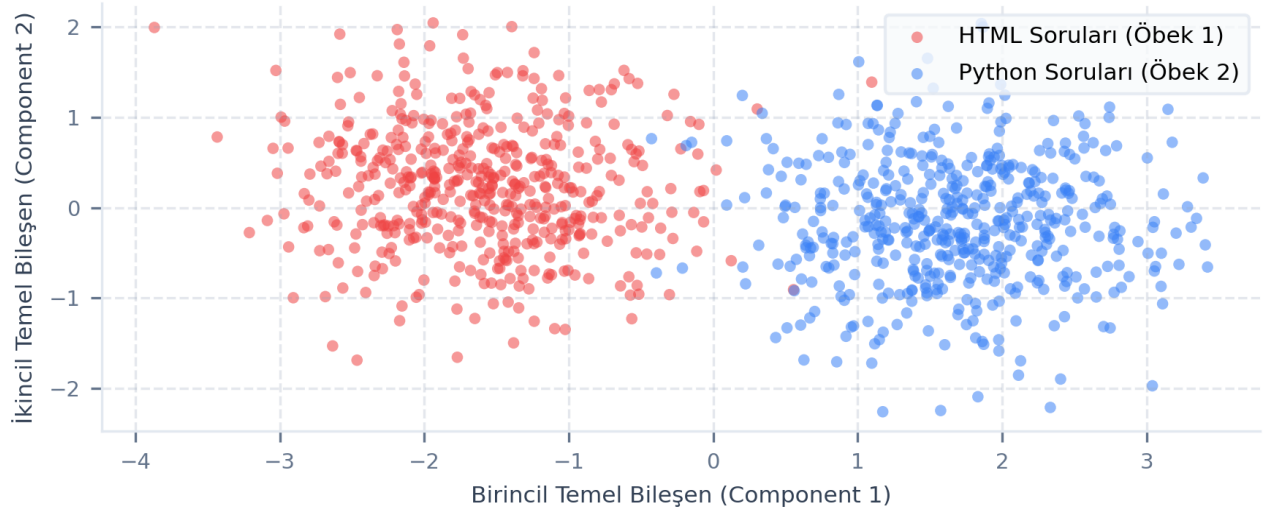
3. Metin Kümeleme (Clustering) ve PCA ile Görselleştirme

Gömme vektörlerinin gücünü göstermenin en doğrudan yolu, etiketi olmayan metinleri gruplandırmaktır. K-Means algoritması kullanılarak ilk 1.000 veri noktası (HTML ve Python soruları) 2 farklı küreye ayrılmıştır. Algoritmaya hangi sorunun hangi dile ait olduğu söylenmemiştir; algoritma sadece 768 boyutlu gömme vektörlerini analiz ederek benzer anlamsal yapıdaki soruları gruplamıştır. 768 boyutu insan gözüyle göremeyeceğimiz için, PCA (Temel Bileşenler Analizi) algoritması ile bu boyutlar 2 boyuta düşürülmüş ve grafik üzerine yerleştirilmiştir.

Günlük Hayat Analjisi: Kitap Tasnifi

Kütüphaneye yeni gelen binlerce karışık kitabı hiç okumadan, sadece kapaklarındaki özetlerin kelime yapılarına göre tasnif edip masal kitaplarını bir köşeye, fen kitaplarını diğer köşeye yığan bir robot kütüphaneciye benzer.

Metin Gömme Vektörlerinin PCA ile 2D Kümeleme Analizi



Grafik 1: HTML ve Python sorularının 768 boyutlu embeddings uzayından PCA ile 2D'ye düşürülmüş K-Means kümelemesi.

```
from sklearn.cluster import KMeans from sklearn.decomposition import PCA # HTML ve Python sorularını
içeren ilk 1000 satırın seçilmesi clustering_data = question_embeddings[:1000] kmeans =
KMeans(n_clusters=2, random_state=42) kmeans.fit(clustering_data) labels = kmeans.labels_ # Küme
etiketleri (0 veya 1) # PCA ile Görselleştirme İçin Boyut İndirgeme pca = PCA(n_components=2) pca_data
= pca.fit_transform(clustering_data)
```

4. Anomali Tespiti (Anomaly Detection)

Embeddings verileri sayesinde sadece benzer verileri bulmakla kalmaz, aynı zamanda sistemdeki uyumsuz veya sapan verileri (aykırı değerleri) de tespit edebiliriz. Tamamı yazılımla ilgili olan Stack Overflow veri setine kurabiye yapımıyla ilgili yabancı bir soru eklenmiştir: *'I'm making cookies, but I don't remember the correct ingredient proportions...'*. Scikit-Learn kütüphanesinin **Isolation Forest** algoritması, bu sorunun gömme vektörünü inceleyerek onu anında 'aykırı değer' (outlier) olarak etiketlemiştir (-1 değeri döner). Ayrıca bazı yanlış etiketlenmiş yazılım soruları da bu sayede anomali olarak yakalanabilmektedir.

Günlük Hayat Analjisi: Tezgâhtaki Farklı Meyve

Bir manav tezgahındaki binlerce elma arasına karışmış bir portakalı ya da tamamen farklı yabancı bir cismi (örneğin oyuncak arabayı) saniyeler içinde ayırt etmeye benzer.

```
from sklearn.ensemble import IsolationForest # Yabancı sorunun gömme vektörünün alınması ve ana diziye
eklenmesi baking_embedding = model.get_embeddings([baking_question])[0].values embeddings_array =
np.vstack([question_embeddings, baking_embedding]) # Isolation Forest Modeli ve Tahmin clf =
IsolationForest(random_state=42) predictions = clf.fit_predict(embeddings_array) # Outlier ise -1
döner is_outlier = predictions[-1] == -1 # True döner
```

5. Metin Sınıflandırma (Classification)

Gömme vektörleri, geleneksel denetimli öğrenme algoritmaları için mükemmel birer sayısal öznitelik (feature) sağlar. Bu çalışmada 2.000 yazılım sorusunun gömme vektörleri girdi (X), ait oldukları yazılım dilleri ise hedef etiket (y) olarak belirlenmiştir. Veri seti **%80 eğitim** ve **%20 test** olmak üzere bölünmüş, ardından **Random Forest** sınıflandırıcısı ile eğitilmiştir. Model, test setindeki soruların hangi programlama diline ait olduğunu **%70 başarı oranı (accuracy)** ile tahmin etmiştir.

Günlük Hayat Analjisi: Akıllı Posta Ayırıştırma

Bir şirkete gelen e-postaları içeriğine göre otomatik olarak ilgili departmanlara (Fatura ise Muhasebe'ye, başvuru ise İK'ya, hata bildirimi ise Teknik Destek'e) doğru şekilde yönlendiren akıllı sekreter sistemine benzer.

```
from sklearn.ensemble import RandomForestClassifier from sklearn.model_selection import
train_test_split from sklearn.metrics import accuracy_score # Eğitim ve test setlerine ayırma X_train,
X_test, y_train, y_test = train_test_split( embeddings_data, df['category'], test_size=0.2,
random_state=42, shuffle=True ) # Sınıflandırma Modelinin Eğitimi clf =
RandomForestClassifier(n_estimators=200, random_state=42) clf.fit(X_train, y_train) preds =
clf.predict(X_test) accuracy = accuracy_score(y_test, preds) # Çıktı: ~0.70
```

6. Metin Gömme Uygulamaları Karşılaştırma Matrisi

Uygulama Alanı	Kullanılan Algoritma	Temel Amaç	Karar Mekanizması
Kümeleme (Clustering)	K-Means + PCA	Grup etiketleri olmayan verileri anlamsal benzerliklerine göre gruplamak.	Vektör uzayındaki mesafeler (Öklid vb.)

Uygulama Alanı	Kullanılan Algoritma	Temel Amaç	Karar Mekanizması
Anomali Tespiti	Isolation Forest	Veri kümesinin genel dağılımına uymayan aykırı metinleri bulmak.	Uzaklık ve ağaç dallanma derinliği
Sınıflandırma	Random Forest	Metinleri önceden belirlenmiş belirli sınıflara/kategorilere atamak.	Ağaçların çoğunluk oylaması (%70 doğruluk)